

SCR2043 OPERATING SYSTEMS

Name : Chau Ying Jia
Student ID : A23CS0213
Section : 02

Marks

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. Download necessary source codes:

Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. Compile the source files:

Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. Execute the dummy processes:

Run all the dummy processes

```
./mainprocess &
```

Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command	
<code>ps -e</code>	
Output	
<pre>1149 pts/0 00:00:00 mainprocess 1150 pts/0 00:00:00 mainprocess 1151 pts/0 00:00:00 mainprocess 1152 pts/0 00:00:00 subprocess1 1153 pts/0 00:00:00 subprocess2 1154 pts/0 00:00:00 subprocess1 1155 pts/0 00:00:00 subprocess2 1156 pts/0 00:00:00 subprocess2 1157 ? 00:00:00 kworker/u5:4-events_power_efficient 1158 pts/0 00:00:00 ps</pre>	

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command	
<code>ps -ef grep -E 'mainprocess subprocess'</code>	
Output	
<pre>chau 1149 1073 0 06:47 pts/0 00:00:00 ./mainprocess chau 1150 1149 0 06:47 pts/0 00:00:00 ./mainprocess chau 1151 1149 0 06:47 pts/0 00:00:00 ./mainprocess chau 1152 1150 0 06:47 pts/0 00:00:00 ./subprocess1 chau 1153 1151 0 06:47 pts/0 00:00:00 ./subprocess2 chau 1154 1150 0 06:47 pts/0 00:00:00 ./subprocess1 chau 1155 1151 0 06:47 pts/0 00:00:00 ./subprocess2 chau 1156 1151 0 06:47 pts/0 00:00:00 ./subprocess2 chau 1271 1073 0 06:49 pts/0 00:00:00 grep --color=auto -E mainprocess subprocess</pre>	

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command										
<code>ps -ef grep subprocess</code>										
Output										
chau	1152	1150	0	06:47	pts/0	00:00:00	./subprocess1			
chau	1153	1151	0	06:47	pts/0	00:00:00	./subprocess2			
chau	1154	1150	0	06:47	pts/0	00:00:00	./subprocess1			
chau	1155	1151	0	06:47	pts/0	00:00:00	./subprocess2			
chau	1156	1151	0	06:47	pts/0	00:00:00	./subprocess2			
chau	2591	1073	0	06:50	pts/0	00:00:00	grep --color=auto subprocess			

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command										
<code>ps -eo pid,user,pcpu,pmem,cmd grep subprocess</code>										

Output										
1152	chau	0.0	0.1	./subprocess1						
1153	chau	0.0	0.1	./subprocess2						
1154	chau	0.0	0.1	./subprocess1						
1155	chau	0.0	0.1	./subprocess2						
1156	chau	0.0	0.1	./subprocess2						
2593	chau	0.0	0.1	grep --color=auto subprocess						

Question 5

Building on the `ps` command used in Question 4, can you add an option to sort the listed processes by their memory usage (`pmem`)?

Command
<code>ps -eo pid,user,pcpu,pmem,cmd --sort=pmem grep subprocess</code>
Output
<pre>1152 chau 0.0 0.1 ./subprocess1 1153 chau 0.0 0.1 ./subprocess2 1154 chau 0.0 0.1 ./subprocess1 1155 chau 0.0 0.1 ./subprocess2 1156 chau 0.0 0.1 ./subprocess2 2596 chau 0.0 0.1 grep --color=auto subprocess</pre>

Question 6

Construct a command using `ps`, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command
<code>ps -eo pid,ppid,cmd --forest grep -E 'mainprocess subprocess1 subprocess2'</code>
Output
<pre>1149 1073 _ ./mainprocess 1150 1149 _ ./mainprocess 1152 1150 _ ./subprocess1 1154 1150 _ ./subprocess1 1151 1149 _ ./mainprocess 1153 1151 _ ./subprocess2 1155 1151 _ ./subprocess2 1156 1151 _ ./subprocess2 2598 1073 _ grep --color=auto -E mainprocess subprocess1 subprocess2</pre>

Question 7

Use `pstree` command with option that show the number of threads to each process.

Command
<code>pstree -c -s 1149</code>
Output
<pre>systemd---sshd---sshd---sshd---bash---mainprocess--+-mainprocess--+-subprocess1 \-subprocess1 \-mainprocess--+-subprocess2 -subprocess2 \-subprocess2</pre>

Question 8

Use `renice` command to change priority level of one of process “subprocess1”.

Command
<code>sudo renice -3 1152</code>
Output
<pre>chau@secr2043:~\$ ps -o pid,nice,comm -C subprocess1 PID NI COMMAND 1152 0 subprocess1 1154 0 subprocess1 chau@secr2043:~\$ sudo renice -3 1152 [sudo] password for chau: 1152 (process ID) old priority 0, new priority -3</pre>

Question 9

Terminate all running processes with the name “mainprocess”.

Command
<code>killall -15 mainprocess</code>
Output
<pre>Main process (ID: 1150) received signal: 15. Terminating... Main process (ID: 1149) received signal: 15. Terminating... Main process (ID: 1151) received signal: 15. Terminating... [1]+ Done ./mainprocess</pre>

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

void child_process() {
    printf("Child process with PID: %d\n", getpid());
    int sleep_time = rand() % 5 + 1;
    printf("Child process sleeping for %d seconds\n", sleep_time);
    sleep(sleep_time);
    printf("Child process exiting\n");
}

int main() {
    printf("Parent process with PID: %d\n", getpid());

    //fork a child process
    pid_t pid = fork();

    if (pid == 0) {
        //child process
        child_process();
        exit(0);
    } else if (pid > 0) {
        //parent process
        printf("Parent process waiting for child process to finish\n");
        wait(NULL);
        printf("Parent process exiting\n");
    } else {
        //error
        perror("fork");
        return 1;
    }

    return 0;
}
```

Output:

```
chau@secr2043:~$ nano multiprocessing.c
chau@secr2043:~$ gcc multiprocessing.c -o multiprocessing
chau@secr2043:~$ ./multiprocessing
Parent process with PID: 2648
Parent process waiting for child process to finish
Child process with PID: 2649
Child process sleeping for 4 seconds
Child process exiting
Parent process exiting
```